FPT Education

TRƯỜNG ĐẠI HỌC FPT

Module 2 - Dart Essentials

Contents

- Structure
- Data types
- Variables
- Collections
- Control Flow
- OOP
- Null Safety
- Async

What is Dart?

Concept

- Dart is a client-optimized, class-based language by Google.

Key Points

- Designed for UI frameworks (Flutter)
- Multi-platform: mobile (iOS/Android), web, and desktop applications
- Fast development: JIT
- Fast production: AOT
- Modern asynchronous model (Futures, async/await, Streams)

Notes: Relate Dart vs JavaScript vs Kotlin for context

Program Structure

- Every Dart program starts at the main() function.
- print() writes a line to the console.

```
void main() {

  print("Hello Dart");

}
```

→ Hello Dart

Data Types

Type	Description
Numbers	int (whole numbers), double (fractional)
String	Text enclosed in single or double quotes
bool	Boolean value: true or false
List	Ordered collection of elements
Set	Unordered collection of **unique** elements
Map	Key–value pairs
null	Represents absence of a value
Object	Base class for all Dart objects
enum	Fixed list of constant values
Iterable	Abstract collection type used in loops
Never	For functions that never return (always throw)
Dynamic	Can hold any type; resolved at runtime
Void	Indicates no return value

Data Types

- Core types: int, double, String, bool, List, Set, Map
- Strong typing with generics (List<int>, Map<String, int>)

```
int age = 20;
double gpa = 3.75;
String campus = "FPT HCM";
bool pass = true;
```

Operators

Concept

- Operators allow you to perform mathematical, comparison, and logical operations.

Explanation

- Arithmetic: + - * / %
- Comparison: > < >= <= ==
- Logical: && || !

```
int a = 5, b = 2;
print(a + b);
print(a > b);
```

Variables & Memory Concept

- var = inferred type, dynamic = runtime type.
- final = runtime-constant (assigned once)
- const = compile-time constant

```
var name = "FPT University";
final year = 2024;
const pi = 3.14159;
dynamic x = 10; x = "Hello";
```

Strings & Interpolation

- Use $var or ${expr} to interpolate values.

```
String name = "FPT University";

print("Hello $name");

print("Length: ${name.length}");
```

```
Hello FPT University
Length: 16
```

Collections (List, Set, Map)

- List: ordered • Set: unique • Map: key-value

```
List<int> nums = [1,2,3];
Set<int> s = {1,2,2,3};
var user = {"id":1, "name":"Nam"};
print(user["name"]);
```

- Output: {1,2,3}

  Nam

Control Flow: if / else - switch

- If/else: Conditional branching with boolean expressions.

```
var day = "Mon";

switch(day) {
 case "Mon": print("Start"); break;
 default: print("Other");
}
```

- Switch is clearer than many if-else for discrete cases.

```
int score = 80;


if(score >= 50) print("Pass");
else print("Fail");
```

Loops: for-in & forEach

- Classic counted loop.

```
for(int i=0; i<3; i++) {
 print(i);
}
```

- for-in: iterate collections • forEach: functional style

```
for(var x in [1,2,3]) print(x);

[1,2,3].forEach((e)=>print(e));
```

Functions: basics - arrow syntax

- Typed params & return values.

```
int add(int a,int b){
  return a+b;
}
print(add(2,3));
```

- Short form for single-expression functions.

```
int add(int a,int b)=>a+b;

print(add(2,3));
```

Functions — optional & named params

- Use {} for named optionals; provide defaults.

```
void greet({String name = "Guest"}) { print(name); }

greet(); // Guest

greet(name: "Minh"); // Minh
```

- Output: `Guest`
  `Minh`

Intro to OOP

Concept

- Dart fully supports OOP

Key ideas

- Class / Object
- Constructor
- Inheritance
- Polymorphism
- Method Override

Notes

OOP will be the foundation when building the Flutter widget tree UI.

Classes & Objects

- Class defines structure; object is an instance.

```
class Car {
 String brand;
 Car(this.brand);
 void start(){ print("Started: $brand"); }
}
var c = Car("Toyota");
c.start();
```

Started: Toyota

Constructors - default & named

- Default, named, and redirecting constructors.

```
class Point {
 int x,y;
 Point(this.x,this.y);
 Point.origin(): x=0, y=0;
}

var p = Point.origin();
```

Inheritance & Override

- **Use extends to inherit and @override to change behavior.**

```
class Animal {
      void sound()=>"Some";
}

class Dog extends Animal {
 @override void sound()=>"Woof";
}

Dog()..sound();
```

Null Safety - operators ?, ??, !

- Dart enforces null-safety at compile-time to prevent null reference errors.

Operator	Meaning
`?`	Declares a nullable variable
`??`	Default fallback if null
`!`	Force non-null assertion

- Example:

```
String? name;          // nullable variable
print(name);           // prints null

print(name ?? "Guest"); // default value if null

name = "Minh";
print(name!.length);   // force non-null after assignment
```

Collections - map/filter

Concept: Functional operations over iterables (e.g., map, where)

- map: transform each element → new Iterable
- where (filter): keep elements that satisfy a predicate
- Often chained; lazy until consumed (toList, forEach, etc.)

Explanation

- Works on any Iterable<T> (e.g., List, Set)
- map preserves length, transforms type
- where preserves type, may change length
- Use toList() or toSet() to materialize results

```
final numbers = [1, 2, 3, 4, 5];

// map: square each number
final squares = numbers.map((n) => n * 2);

// filter: keep even numbers
final evens = numbers.where((n) => n.isEven);

// chain: square then keep > 5
final big = numbers.map((n) => n * 2).where((n) => n > 5);

// materialize
print(squares.toList());
print(evens.toList());
print(big.toList());

// map can change type
final labels = numbers.map((n) => 'Item $n').toList();
print(labels);
```

Exceptions — try/catch/finally

Concept: Handle runtime exceptions without crashing the application.

- try: the risky block
- catch / on: handle exceptions
- finally: always runs (cleanup)
- rethrow to propagate after logging

Explanation

- Use on SpecificException to target types
- catch (e, s) captures error + stack trace
- Prefer throwing custom exceptions for domain errors

Example:

```
int parsePositiveInt(String s) {
  final value = int.parse(s); // may throw FormatException
  if (value < 0) throw ArgumentError('Must be >= 0');
  return value;
}

void main() {
  try {
    print(parsePositiveInt("12"));
    print(parsePositiveInt("-3"));      // throws ArgumentError
  } on FormatException catch (e, stack) {
    print("Format error: $e");
  } on ArgumentError catch (e) {
    print("Argument error: ${e.message}");
  } catch (e, stack) {
    print("Unknown error: $e");
    // rethrow; // (optional) bubble up
  } finally {
    print("Done parsing.");
  }
}
```

Async — Future & async/await

- async marks an async function; await pauses until Future completes.

```
Future<String> fetch() async {
 await Future.delayed(Duration(seconds:1));
 return "Done";
}

void main() async {
 print(await fetch());
}
```

Streams — sequence of async values

Exam:

- A Stream emits a sequence of asynchronous values over time.
- Single-subscription (default): one listener, sequential consumption.
- Broadcast: multiple listeners, events shared.

Explanation:

- Subscribe with listen, cancel with subscription.cancel().
- Transform with **map / where / take / debounce (package)`.
- Consume with await for inside async functions.

```
// Example 1: periodic stream (single-subscription)
final stream = Stream.periodic(const Duration(milliseconds: 400), (i) => i)
    .take(5); // 0..4 then complete

stream.listen(
  (value) => print('Value: $value'),
  onDone: () => print('Done'),
);

// Example 2: transform & filter
final evens = stream.where((n) => n.isEven).map((n) => n * 10);
evens.listen((v) => print('Even*10: $v'));
```

Mini-lab - Student list

- Task: create Student class, store in List, print, then add async delay to simulate loading.

```
class Student { String name; Student(this.name);}
void main() async {
 var list = [Student("An"), Student("Binh")];
 await Future.delayed(Duration(seconds:1));
 list.forEach((s)=>print(s.name));
}
```

Summary

We covered:

- Dart structure
- data types -Variables
- Collections
- Control flow
- Functions
- OOP
- Null safety
- Async programming and streams.

References

- Mastering Flutter (2025), Chapter 2
- docs.flutter.dev
- dart.dev/tools/dartpad